

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Engineering (CSE)

MID SEMESTER EXAMINATION
DURATION: 1 HOUR 30 MINUTES

SUMMER SEMESTER, 2024–2025
FULL MARKS: 75

CSE 4835: Pattern Recognition

Programmable calculators are not allowed. Do not write anything on the question paper.

Answer **all 3 (three)** questions. Figures in the right margin indicate full marks of questions with corresponding COs and POs in parentheses. Appendix provided on Page 3.

1. a) Assume your teacher set this semester's *Pattern Recognition* Mid-Semester exam by choosing each question **verbatim** from a bank of N past questions with known answers. Three students answer using a KNN model: for each question they retrieve the k most similar questions from the bank and reply with the majority answer. Their values of k are 1, $\frac{N}{2}$, and N , respectively. Rank the three students by expected exam score, and justify your ranking.

5
(CO1)
(PO1)

Solution:

Best to worst: $1 > \frac{N}{2} > N$.

KNN with $N = 1$ memorizes the training data. Since the question is chosen verbatim from the previous questions, memorization will have a 100% score. As the value of N increases, it matches with more and more unrelated questions, and the score gradually decreases. The expected score therefore will be lower for $\frac{N}{2}$ and lowest for N .

- b) For each statement, state whether it is **True** or **False** and justify your answer briefly.
- If a weight matrix W gives a multiclass SVM loss (margin $\Delta = 1$) of exactly 0 on the training set, then $\frac{W}{2}$ also gives zero loss on the training set.
 - A model trained on CIFAR-10 (ten equally represented classes in both the training and test sets), with every training label accidentally set to “class 1”, will have 0% test accuracy.
 - Stochastic gradient descent always reaches the same final loss as full-batch gradient descent.
 - Gradient descent can reduce the training loss of a neural network only if the network contains non-linear layers.
 - Adding a validation split *guarantees* equal or better test accuracy compared to using just a train-test split.

5×3
(CO1)
(PO1)

Solution:

(i) **False** — halving W halves every score gap $s_j - s_y$, so margins between 1 and 2 drop below $\Delta = 1$ and incur loss.

(ii) **False** — the model predicts “class 1” for everything, scoring about 10% on a balanced test set, not 0%.

(iii) **False** — no such guarantee; SGD is noisier and may hover, while full-batch GD can reach a lower loss.

(iv) **False** — GD optimises any network.

(v) **False** — Using a train-test split and choosing the best model accordingly will indirectly overfit on the test set, therefore giving you better test performance.

- c) You train a model with mini-batches on a dataset sorted by class label (all examples of one class, then all of the next, and so on), *without shuffling*. The batches are small relative to the number of examples per class. Describe how the training loss will behave over an epoch.

5
(CO1)
(PO1)

Solution:

Sorted data makes each small batch (almost) one class, so each step is biased toward that class; the model chases one class then the next, oscillating.

2. a) Figure 1 shows, as an image, the weights a linear classifier learned for the handwritten digit “7” (darker = larger weight).

4 + 4
(CO2)
(PO2)

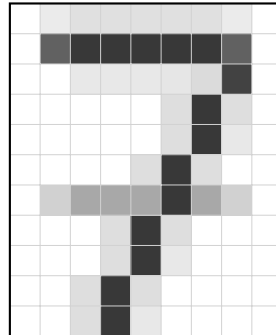


Figure 1: Weights learned by a linear classifier for the handwritten digit “7” (Question 2.a).

- i. Besides the two main strokes of the “7” (the top bar and the diagonal), a *faint* horizontal stroke crosses the middle. Account for why it occurs, and explain why it is faint.

Solution:

Linear classifier stores one weight image per class, effectively the average of all training “7”s. Some are written with a crossbar and some without, so the average superimposes both styles; because only some have the crossbar, it is faint.

- ii. Explain how a neural network with a hidden layer represents the class differently, and which limitation of the linear classifier this overcomes.

Solution:

A hidden layer lets the network learn several low-level features (hidden units) — e.g. one tuned to the top-bar, one tuned to the middle-bar, one tuned to the diagonal line and combine them downstream. The class is then represented by several prototypes rather than one, capturing the within-class variation that a single linear score cannot.

- b) You visualise a trained CNN’s *first convolutional layer* filters and notice that every filter looks like a local edge or low-level feature, never a whole object.

4 + 4
(CO3)
(PO3)

- i. Explain why the filters in the first convolutional layer must take this local form.

Solution:

Each first-layer filter sees only a small $K \times K$ receptive field, so it can only learn a structure that fits in that patch.

- ii. Is it possible to design a first convolutional layer whose filters respond to a whole object? If yes, give the design; if not, explain why it is impossible.

Solution:

Yes: set the kernel size equal to the input size. The filter then spans the whole image and can encode a whole-object template. (Further question: *why is this bad?*)

- c) A model's weights start far from the loss minima. As it descends, the gradient keeps the same sign and grows in magnitude from step to step.

6 + 3
(CO2)
(PO2)

- i. Describe how the size of each update evolves under (1) SGD with momentum, (2) RMSProp, and (3) Adam, and state which descends to the minima fastest.

Solution:

Momentum is fastest. Its velocity is a running sum of consistent, growing gradients, so it grows even faster than g_t itself and the step keeps accelerating.

RMSProp divides the step by $\sqrt{G_t} \propto |g_t|$, so the magnitude cancels and the step holds a fixed $\approx \alpha$ scale however large g_t becomes.

Adam converges slower than *Momentum* but faster than *RMSProp*: the Momentum term in Adam pushes it to converge faster, while the RMSProp term slows it down. A combination of both makes it behave like regular gradient descent without any optimizer.

- ii. A student mistakenly writes Adam's update without the square root: $w_{t+1} = w_t - \frac{\alpha \hat{m}_t}{\hat{v}_t + \epsilon}$. Describe how this update behaves, and how it differs from regular Adam.

Solution:

With $\hat{m}_t \approx g_t$ and $\hat{v}_t \approx g_t^2$, the step becomes $\approx \frac{\alpha g_t}{g_t^2} = \frac{\alpha}{g_t}$: its size is now *inversely* proportional to the gradient magnitude, so it shrinks toward zero.

3. a) A three-class classifier (classes A, B, C) produces the scores $\mathbf{s} = (s_A, s_B, s_C) = (2, 0, 3)$ for a training example whose correct class is A . Use margin $\Delta = 1$ for the SVM loss.

3 +
4 + 6
(CO1)
(PO1)

- i. Compute the multiclass SVM loss for this example.

Solution:

$$\max(0, 0 - 2 + 1) + \max(0, 3 - 2 + 1) = 0 + 2 = 2.$$

- ii. Compute the softmax cross-entropy loss for this example.

Solution:

$$-\log \frac{e^2}{e^2 + e^0 + e^3} = -\log \frac{7.389}{28.475} = -\log(0.2595) \approx \mathbf{0.586}.$$

- iii. After further training, the scores for this same example become $(5, 0, 1)$. Recompute both losses, and state which loss you would prefer to keep training with here, and why.

Solution:

New scores $(5, 0, 1)$:

$$\text{SVM} = \max(0, -4) + \max(0, -3) = \mathbf{0};$$

$$\text{CE} = -\log \frac{e^5}{e^5 + 1 + e^1} = -\log(0.9756) \approx \mathbf{0.0108}.$$

Only the cross-entropy loss still has a non-zero gradient. SVM loss is exactly 0 once

all margins are met, so it gives no further gradients; cross-entropy is never exactly 0 and keeps adjusting.

- b) Figure 2 shows the computational graph of $f(x, y, z) = \frac{1}{\max(xy, z)}$ with intermediate nodes $a = xy$, $m = \max(a, z)$, and $f = \frac{1}{m}$. Evaluate using $x = 3$, $y = 2$, $z = 4$.

12
(CO1)
(PO1)

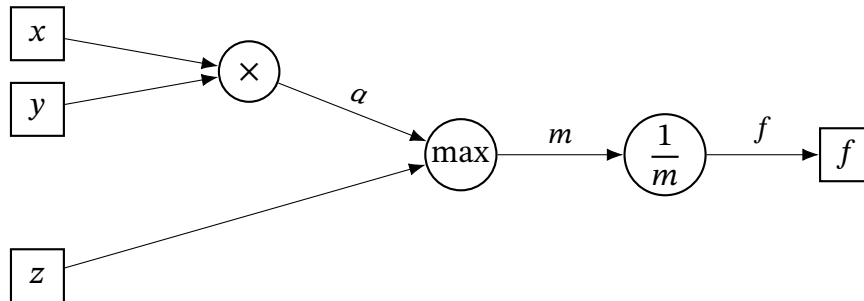


Figure 2: Computational graph for Question 3.b.

Compute the forward-pass values a , m , f , and the backward-pass gradients $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$, $\frac{\partial f}{\partial z}$.

Solution:

(b) Forward: $a = xy = 6$, $m = \max(6, 4) = 6$, $f = \frac{1}{m} = \frac{1}{6}$.

Backward: $\frac{\partial f}{\partial m} = -\frac{1}{m^2} = -\frac{1}{36}$.

The max passes the gradient only to its larger input: since $a = 6 > z = 4$,

$\frac{\partial m}{\partial a} = 1$ and $\frac{\partial m}{\partial z} = 0$.

With $\frac{\partial a}{\partial x} = y = 2$ and $\frac{\partial a}{\partial y} = x = 3$:

$$\frac{\partial f}{\partial x} = -\frac{1}{36} \cdot 2 = -\frac{1}{18}, \quad \frac{\partial f}{\partial y} = -\frac{1}{36} \cdot 3 = -\frac{1}{12}, \quad \frac{\partial f}{\partial z} = -\frac{1}{36} \cdot 0 = 0.$$

MARKS SUMMARY TABLE

Question	Marks	Keywords
1.a	5	
1.b	15	
1.c	5	
2.a	8	
2.b	8	
2.c	9	
3.a	13	
3.b	12	
Total	75	